MTP STAGE-2 REPORT

Low-Cost Automated Tomato Sorting System



Submitted by

Jay Sonawane

M.Tech, Computer Science and Engineering

Under the Guidance of

Prof. Shivaram Kalyanakrishnan

Contents

1 Introduction **3**

2 Recap of MTP Stage-1 **5**

2.1 Problem and Hardware Setup 5

2.2 Model Iterations 5

2.3 Automated Annotation with SAM 6

2.4 Mobile Deployment 6

3 Literature Survey **7**

4 Diagnosing the Stage-1 Model's Failure on Chamber Data **8**

4.1 Initial Observation: Live Chamber Failure 8

4.2 Ruling Out Model Capacity 8

4.3 Ruling Out Background Alone 9

4.4 Identifying the Actual Cause: a Colour-Lighting Mismatch 9

4.5 Localization as a Compounding Factor 10

4.6 Summary of the Diagnosis 10

5 Chamber Dataset Collection and Annotation **12**

5.1 Capture Protocol 12

5.2 Automated Full-Frame Annotation using SAM2 13

5.3 Chamber Lighting as a Dataset Variable 14

6 Model Fine-Tuning **15**

6.1 Train/Validation Split by Tomato 15

6.2 Training Configuration 15

7 Evaluation and Generalization Testing **18**

7.1 The Generalization Gap 18

7.2 Diagnosis and Correction 18

7.3 Final Held-Out Results 19

7.4 Interpreting These Results 19

8 Edge Deployment on Raspberry Pi **21**
8.1 Hardware Selection 21
8.2 Model Export and Parity Verification 21
8.3 Thread-Count Tuning 22
8.4 INT8 Quantization: Evaluated and Rejected 22
8.5 Event-Triggered Inference 23

9 Results **24**
9.1 Diagnosis of the Stage-1 Model 24
9.2 Final Model Performance 24
9.3 Edge Deployment Performance 25

10 Conclusion and Future Work **26**

1. Introduction

MTP Stage-1 established a classification pipeline for tomato ripeness using MobileNetV2, demonstrated the effect of background standardization on accuracy, and carried out a preliminary exploration of YOLO-based detection using automatically generated annotations. That pipeline was trained and evaluated on a curated, segmented, single-tomato image dataset.

MTP Stage-2 moves the system out of that curated setting and into the physical sorting chamber that the classification model will actually operate in. This transition exposed a substantial gap between the Stage-1 model's performance on its own curated test set and its performance on live, real-world chamber footage. A central part of this stage was therefore diagnostic: identifying precisely why a model that reported near-perfect validation metrics in Stage-1 failed when pointed at the real chamber, before any further model development could be trusted.

The specific objectives pursued in this stage are as follows.

- Diagnose the cause of the Stage-1 YOLO model's failure on real chamber footage, through controlled experiments rather than assumption.
- Build an annotation pipeline capable of labelling full, real-background chamber photographs – rather than the segmented, background-removed images used in Stage-1 – so that the resulting model is trained on data that matches its deployment conditions.
- Extend the system to a three-class problem (unripe, ripe, overripe) using real chamber-captured data for all three classes, including overripe, which required sourcing and preparing data despite overripe tomatoes being difficult to procure.
- Fine-tune a YOLO11n detection model on this chamber dataset and validate it against data the model had never seen during training, not only against a held-out split from the same capture session.
- Deploy the resulting model on embedded hardware (a Raspberry Pi) and evaluate whether it can run at a useful speed and with its accuracy preserved outside a full desktop GPU environment.

- Design and validate an event-triggered inference approach, so that the deployed model only performs inference when a tomato is actually present for classification, rather than running continuously.

This report documents the diagnostic process, the chamber data collection and annotation pipeline, the resulting fine-tuned model and its evaluation, and the embedded deployment work carried out on a Raspberry Pi, including a comparison of inference backends and an event-triggered inference design.

2. Recap of MTP Stage-1

This chapter briefly summarizes the system developed in Stage-1, since the diagnostic work in Stage-2 (Chapter 4) is a direct response to limitations discovered in this prior system.

2.1 Problem and Hardware Setup

The underlying problem is the automated classification of tomatoes into ripeness categories (unripe, ripe, overripe) for sorting. Stage-1 developed a vertical cylindrical prototype, built with the e-Yantra lab, consisting of a single camera paired with a mirror so that one captured frame contains both a direct view and a mirrored view of the tomato simultaneously, a roller mechanism to rotate the tomato for multi-view capture, and an Arduino-controlled feeder and sorting mechanism.

2.2 Model Iterations

Three modelling approaches were explored in Stage-1.

Model A trained a MobileNetV2 classifier directly on raw, unprocessed tomato images. This model reached approximately 89% validation accuracy, but analysis showed it was partly learning background-specific features rather than tomato-specific ones, which caused misclassification when the background varied.

Model B addressed this by standardizing the background: a saturation threshold was used to detect the tomato region, and everything outside it was replaced with solid black before training. This raised accuracy to approximately 97%, and this background-standardization principle – removing background variation from the training images – became the basis for the automated annotation pipeline described next. It also, as later diagnosed in Stage-2, introduced a background-dependence problem of its own.

Model C explored a classical bounding-box localization approach combining edge detection and GrabCut-based segmentation. This was investigated for visualization purposes only and was not taken forward for training.

2.3 Automated Annotation with SAM

To move from classification toward detection, Stage-1 used the Segment Anything Model (SAM2) in automatic mask-generation mode to segment each curated tomato image, producing a clean object mask without manual annotation. The non-background pixels of this mask were then composited onto a black canvas, and a bounding box was computed as the extent of the mask. This labelled dataset was used to train a YOLO11s detection model, which reported a validation mAP50-95 of 0.995 – a very high and, as this report's next chapter shows, misleadingly high figure.

2.4 Mobile Deployment

The trained MobileNetV2 classifier (Model B) was converted to TensorFlow Lite and integrated into a Flutter mobile application, enabling on-device inference without requiring a continuous server connection. A probability-fusion scheme averaged softmax outputs across the direct and mirrored views captured by the chamber's camera to produce a single, more robust classification per tomato.

3. Literature Survey

The literature survey conducted at the start of Stage-1 – covering deep learning-based classification systems, YOLO-based detection systems for agricultural sorting, and conveyor-based sorting hardware – remains the relevant background for this stage and is not repeated here. The reader is referred to the Stage-1 report for that survey and its accompanying comparison table of existing systems.

Stage-2's focus shifted from surveying modelling approaches to a diagnostic and systems-engineering problem: understanding why a previously-validated model failed outside its training distribution, and building the infrastructure to correct this and deploy the corrected model on embedded hardware. Accordingly, no additional literature review was undertaken for this stage; the relevant comparison in this stage is empirical, against the project's own Stage-1 results.

4. Diagnosing the Stage-1 Model's Failure on Chamber Data

This chapter presents the central diagnostic investigation of Stage-2: understanding why the Stage-1 YOLO model, despite reporting a validation mAP50-95 of 0.995, failed when evaluated on real footage from the physical sorting chamber.

4.1 Initial Observation: Live Chamber Failure

The Stage-1 YOLO11s model was first evaluated by running it live against the chamber's camera feed. Two failures were observed immediately. First, pointing the camera at a completely empty chamber produced a confident detection (class "unripe", confidence 0.97) where no tomato was present at all. Second, when a genuinely ripe tomato was placed in the chamber, the model frequently misclassified it as unripe or overripe rather than ripe.

This contradicted the model's reported validation performance and indicated that the 0.995 mAP50-95 figure did not reflect how the model would behave in actual deployment.

4.2 Ruling Out Model Capacity

One hypothesis was that YOLO11s, despite being a relatively small model, might still be too large or too small in the wrong way for this task. To test this, a YOLO11n model (approximately one-third the parameter count of YOLO11s) was trained on the identical Stage-1 dataset and evaluated on the same set of real chamber images (96 ripe and 95 unripe crops, captured from the physical chamber).

Table 4.1: Ripe-class accuracy on real chamber images, by model size

Model	**Parameters**	**Ripe accuracy (chamber images)**
YOLO11n	≈2.6M	20.8%
YOLO11s	≈9.4M	18.8%

Both models failed on the ripe class to a similar degree, with the larger model per-

forming marginally worse, not better. This rules out model capacity as the cause of the failure: a model-size explanation would predict the larger model should perform noticeably better, which was not observed.

4.3 Ruling Out Background Alone

A second hypothesis was that the mismatch was purely a background problem: the model was trained on tomatoes against solid black, and the chamber presents tomatoes against grey walls and a mirror, so the model might simply be confused by the presence of a real background. To test this directly, the same real chamber images were re-segmented using SAM2 and composited onto black – replicating the Stage-1 training distribution's background exactly – and re-evaluated.

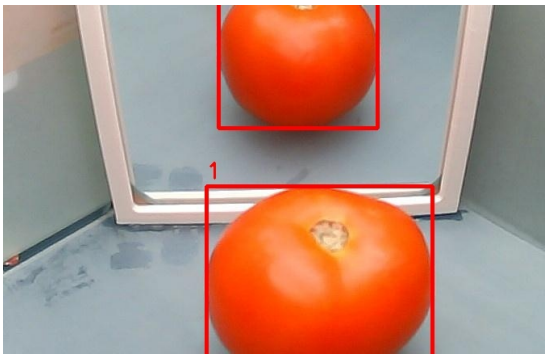
If background mismatch were the entire explanation, restoring a matching black background should have recovered accuracy close to the model's original reported performance. Instead, ripe-class accuracy dropped to 0%, with the model assigning the "overripe" label to every single ripe tomato at 96–98% confidence. This result is inconsistent with a background-only explanation: removing the background mismatch made the failure *more* severe and more confident, not less.

4.4 Identifying the Actual Cause: a Colour-Lighting Mismatch

The consistency and high confidence of the black-background failure suggested the model was not confused, but was doing exactly what it had learned – on data whose colour statistics did not match training. A direct visual comparison of a Stage-1 training image and a chamber-captured image of the same ripeness class supported this explanation.



(a) Stage-1 training image ("ripe")



(b) Chamber-captured ripe tomato

Figure 4.1: Colour comparison between Stage-1 training data and real chamber footage. The Stage-1 dataset's "ripe" class is dominated by a dull orange/amber tone, while the chamber's lighting renders a genuinely ripe tomato as a vivid, saturated red – a colour the model had only ever seen labelled "overripe" during training.

This is consistent with every result above. The unripe class, which is separated from ripe/overripe by a large colour gap (green versus red), remained robust under both the live-chamber test and the black-background re-test, since a lighting shift large enough to matter for a fine distinction within the red spectrum is not large enough to push green into red. The ripe/overripe boundary, which depends on a much finer colour distinction, was the one that collapsed.

4.5 Localization as a Compounding Factor

A further, structurally independent issue was identified in the live-chamber detections: Stage-1 training labels had bounding boxes covering roughly 50–80% of the frame (since the box was defined as the extent of the segmented, black-composited tomato, which was captured to fill most of the frame). In the real chamber feed, two tomato instances (the direct view and its mirror reflection) are each a much smaller fraction of a wider, cluttered frame. This scale and position mismatch is a second, independent reason the Stage-1 model struggled to even locate tomatoes reliably in chamber footage, on top of the colour-classification problem.

4.6 Summary of the Diagnosis

Three controlled checks – model-size comparison, background-only re-test, and direct visual colour comparison – converge on the same conclusion: the Stage-1 YOLO model's black-background, single-object training data caused it to learn a colour-to-label mapping and an object-scale expectation that do not match the chamber's real lighting,

background, and framing. This diagnosis directly motivated the data collection and annotation approach described in the next chapter: training on full, real chamber photographs rather than segmented, background-removed crops.

5. Chamber Dataset Collection and Annotation

This chapter describes the data collection protocol used to capture real chamber images for all three ripeness classes, and the automated annotation pipeline built to label them without requiring manual bounding-box annotation.

5.1 Capture Protocol

Images were captured using the chamber's existing camera and mirror setup, under a fixed white LED lighting preset. For each tomato, six frames were captured at different rotation positions, so that each training example reflects a specific pose of the fruit rather than a single canonical view. Each frame naturally contains two tomato instances – the direct view and the mirror-reflected view – consistent with the chamber's physical design from Stage-1.

Table 5.1: Chamber capture summary by class

Class	**Tomatoes captured**	**Positions/tomato**	**Frames**
Ripe	8 (initial) + 3 (added later)	6	48 + 18
Unripe	7 (after removing 1 bad batch)	6	42
Overripe	8	6	48

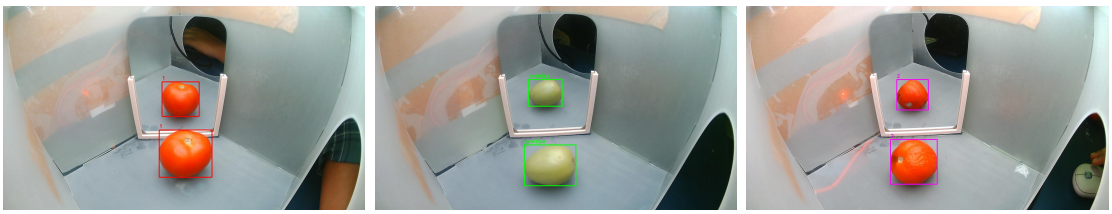
Overripe tomatoes were the most difficult class to source, since vendors do not sell tomatoes at this stage and natural ripening under ambient conditions was unreliable – an earlier attempt to let tomatoes soften naturally resulted in mould on two fruits due to high room temperature and insufficient airflow between tomatoes. The overripe batch used in the final dataset was collected under corrected conditions (tomatoes spaced apart, with airflow, checked daily).

5.2 Automated Full-Frame Annotation using SAM2

Unlike the Stage-1 pipeline, which segmented a single pre-cropped tomato per image, Stage-2 required locating and labelling *two* tomato instances (direct and mirror view) within each full, real-background chamber photograph. The Segment Anything Model (SAM2), in automatic mask-generation mode, was used to propose candidate object masks across each full frame without requiring a manual click or bounding-box prompt per image.

SAM2's automatic mode proposes candidate masks based on general visual coherence (consistent boundaries, texture, and shape), with no built-in notion of "tomato" specifically. Several iterations were required to reliably isolate the two correct tomato instances from the candidates it proposed:

- An initial, unfiltered version occasionally selected the mirror's physical frame or mounting hardware as a candidate object, since these are also visually coherent, well-bounded shapes.
- A colour-based filter was added, retaining only candidate masks whose pixels fell predominantly within a tomato-appropriate hue range (red/orange for ripe and overripe, green/yellow for unripe) and above a minimum saturation threshold. This filter removed the grey mirror-frame and wall false positives.
- The two highest-scoring remaining candidates per frame – scored by a combination of area, bounding-box fill ratio, and proximity to frame centre – were retained as the direct and mirror tomato instances and converted to normalized YOLO-format bounding boxes.



(a) Ripe (b) Unripe (c) Overripe

Figure 5.1: Example SAM2-generated annotations on full, real-background chamber frames, one per class. Both the direct and mirror-view tomato instances are correctly localized in a single automatic pass, with the real chamber background preserved – unlike the Stage-1 pipeline, which discarded the background entirely.

A small number of frames where the automatic pipeline failed to produce two valid instances (for example, where a single tomato's colour signature was weaker than usual,

causing the colour filter to reject it) were identified through visual review and discarded rather than corrected manually, since the discarded fraction was small relative to the overall dataset.

5.3 Chamber Lighting as a Dataset Variable

Because the diagnosis in Chapter 4 established that the model's colour-to-ripeness mapping is sensitive to lighting conditions, the chamber's lighting was treated as a controlled variable throughout data collection: a fixed white preset and brightness level were used for every capture session, and this setting was later re-verified (Chapter 8) after unrelated hardware changes to the chamber's lighting controller, by measuring floor brightness in an empty frame against the mean brightness recorded across the training dataset.

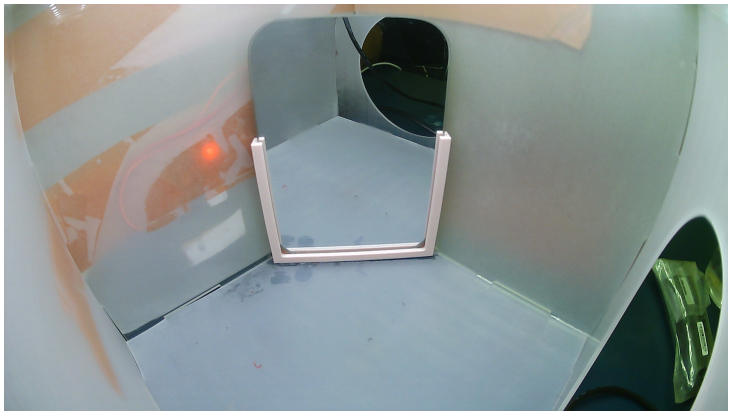


Figure 5.2: Empty chamber under the calibrated white lighting preset used for all data capture, confirmed to match the dataset's measured brightness distribution.

6. Model Fine-Tuning

This chapter describes the training of the final three-class (unripe, ripe, overripe) detection model on the chamber dataset described in Chapter 5.

6.1 Train/Validation Split by Tomato

An important methodological correction was made at this stage. An initial dataset split assigned individual *frames* to train or validation at random, meaning different rotational views of the same physical tomato could appear in both the training set and the validation set simultaneously. Since the six views of one tomato are highly similar, this allows a model to score well on validation simply by having already seen that same fruit (from a different angle) during training – inflating the validation metric without testing genuine generalization.

This was corrected by splitting at the level of the *tomato*, not the frame: all six views of a given tomato are assigned entirely to either the training set or the validation set, never split across both.

Table 6.1: Final dataset split, by tomato

Class	**Tomatoes (train / val)**	**Train frames**	**Val frames**
Unripe	6 / 1	36	6
Ripe	9 / 2	54	12
Overripe	7 / 1	42	6

6.2 Training Configuration

A YOLO11n model was fine-tuned on this dataset at an input resolution of 640×640, for 80 epochs. Two initializations were compared: continuing from the project's existing two-class (ripe/unripe) chamber-trained weights, versus training fresh from COCO-pretrained weights. Both variants converged to comparable validation performance; the COCO-initialized model was ultimately selected as the final model, as it produced cleaner, less hedged predictions on the ripe class and did not depend on an earlier training run whose

own validation split had partially overlapped with tomatoes later used for testing.

Table 6.2: Final model validation metrics (same-session held-out split), epoch 80

Metric	**Value**
Precision	0.981
Recall	0.986
mAP50	0.981
mAP50-95	0.944

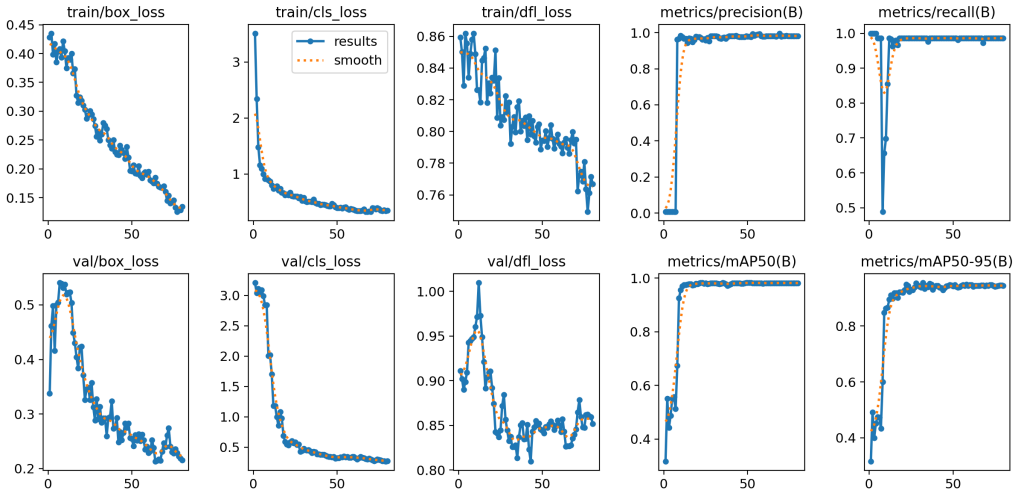


Figure 6.1: Training and validation loss/metric curves for the final three-class model.

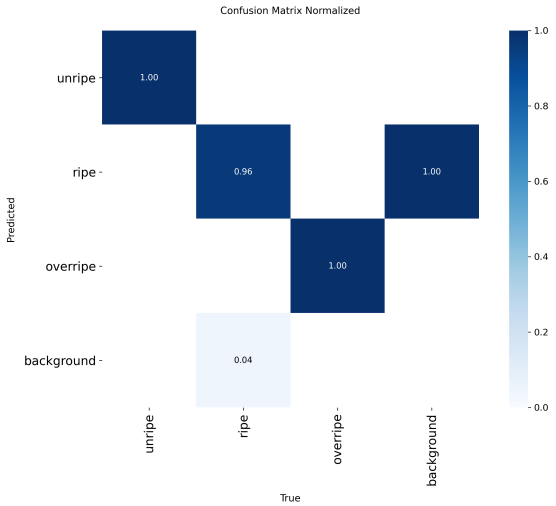


Figure 6.2: Normalized confusion matrix on the same-session held-out validation split.

As established in Chapter 4, however, a same-session validation split – even one correctly separated by tomato – is still not sufficient evidence that a model generalizes

beyond the specific conditions of one capture session. This question is addressed directly in the next chapter.

7. Evaluation and Generalization Testing

7.1 The Generalization Gap

To test whether the fine-tuned model generalized beyond its own training session, a separate test set was captured on a different day: 6 ripe and 6 unripe tomatoes, none of which had been used in training, captured under the same lighting protocol but in an independent session.

The first run of this test produced a clear failure: on the held-out ripe tomatoes, the model classified **0 of 6** tomatoes correctly, with every ripe tomato misclassified as overripe at approximately 0.96 confidence. Unripe tomatoes from the same new session were classified correctly on 6 of 6. This confirmed that even the corrected, real-chamber-trained model had not yet generalized across sessions for the ripe class specifically – the same fragile ripe/overripe boundary identified in Chapter 4, now reappearing in a new form.

7.2 Diagnosis and Correction

The most plausible explanation is that the ripe class, trained on only 8 tomatoes from a single batch and session, had not seen enough natural variation in ripe tomato appearance to generalize to a new batch. To address this directly, three of the six new-session ripe tomatoes were added to the training set (bringing the ripe class to 11 tomatoes total, as reflected in Chapter 6's final split), and the remaining three were kept as a genuinely held-out generalization test.

7.3 Final Held-Out Results

Table 7.1: Held-out, cross-session test results (frames correctly classified as top-ranked class / total; tomatoes classified correctly / total)

Class	Frame accuracy	Tomato accuracy
Ripe (3 held-out tomatoes)	18/18	3/3
Unripe (6 held-out tomatoes)	34/36	6/6
Overripe	*[to be inserted]*	*[to be inserted]*

Overripe generalization testing was carried out following the same cross-session protocol as ripe and unripe above; the corresponding results will be inserted here once finalized.

Per-tomato classification is computed by summing the model's confidence score for each class across all six captured views of that tomato and selecting the class with the highest total – the same multi-view fusion principle used for the mobile application in Stage-1, now applied to the detection model's confidence outputs rather than a plain classifier's softmax vector.



Figure 7.1: Example model predictions on a held-out validation batch, showing correctly localized and classified direct and mirror-view tomato instances.

7.4 Interpreting These Results

The ripe-class failure and recovery illustrates a broader point relevant to the rest of this project: a model's reported validation accuracy is only informative to the extent that the validation data is genuinely independent of training, both at the level of individual

images and at the level of the underlying batch or session the data was drawn from. The same-session 0.944 mAP50-95 reported in Chapter 6 did not predict this failure; the cross-session test did. This result is treated as the primary evidence of the final model's real-world performance, rather than the same-session validation metric alone.

8. Edge Deployment on Raspberry Pi

This chapter describes the deployment of the final fine-tuned model onto a Raspberry Pi 4, including backend comparison, performance benchmarking, and the design of an event-triggered inference approach.

8.1 Hardware Selection

Two single-board computers were considered. A Raspberry Pi Zero W, initially available, was assessed by specification (a single-core, 1 GHz ARMv6 processor with 512 MB of RAM) to be unable to run the required inference stack at all, since no current PyTorch build targets its processor architecture; this was not empirically tested, as the only available SD card for this board failed during setup. A Raspberry Pi 4 Model B (8 GB RAM, quad-core 64-bit ARM Cortex-A72) was used instead and is the hardware this chapter's results are based on.

8.2 Model Export and Parity Verification

The final PyTorch model was exported to NCNN format, a lightweight C++ inference library designed for ARM CPUs without GPU acceleration, at full (FP32) precision. Before trusting any Pi-based results, the exported model's predictions were directly compared against the original PyTorch model on seven held-out test frames, to confirm the export process had not altered the model's behaviour.

Table 8.1: PC (PyTorch, GPU) vs. Raspberry Pi (NCNN, CPU) – prediction parity on 7 held-out frames

	PC (PyTorch)	**Raspberry Pi 4 (NCNN)**
Classes predicted	all 7 correct	all 7 correct, identical classes
Confidence agreement	–	matched to within 0.01 on 6/7 frames
Mean latency	16 ms	389 ms

All seven frames produced identical top-ranked classifications on both platforms, confirming the exported model preserves the PyTorch model's behaviour; the only material

difference is inference speed, which is expected given the absence of a GPU on the embedded board.

8.3 Thread-Count Tuning

NCNN's inference speed on a multi-core ARM CPU depends on the number of CPU threads it is permitted to use. This was benchmarked directly, bypassing the higher-level Python wrapper (which did not expose this setting reliably) and calling the NCNN library directly.

Table 8.2: NCNN inference latency vs. CPU thread count, Raspberry Pi 4

Threads	**Mean latency**	**FPS**
1	680 ms	1.47
2	440 ms	2.27
3	405 ms	2.47
4	362 ms	2.76

Using all four of the Pi's CPU cores very nearly doubles throughput compared to a single thread, with diminishing returns beyond two threads. 362 ms per inference, using all four cores, is taken as the practical latency figure for this hardware.

8.4 INT8 Quantization: Evaluated and Rejected

As a further optimization, the model was additionally exported to TensorFlow Lite with full INT8 quantization, which reduces both model size and, in principle, computation by representing weights and activations as 8-bit integers rather than 32-bit floats. This was benchmarked under the same conditions as the NCNN FP32 model, using the same seven held-out frames.

Table 8.3: NCNN FP32 vs. TFLite INT8 on Raspberry Pi 4

	NCNN (FP32)	**TFLite (INT8)**
Mean latency	362 ms	484 ms
Ripe classification	correct on all held-out frames	incorrectly ranked "overripe" above "ripe" on one held-out ripe frame

INT8 quantization was both slower in practice on this hardware (likely due to the available inference runtime's optimization for this specific board being weaker than NCNN's

hand-tuned ARM kernels, despite INT8's theoretical compute advantage) and less accurate, specifically on the already-fragile ripe/overripe boundary identified in Chapter 4. This is plausibly compounded by the limited calibration set available for quantization (24 images, against a generally recommended minimum of around 300). On this basis, INT8 quantization was not adopted, and the NCNN FP32 export remains the deployed model.

8.5 Event-Triggered Inference

A specific design requirement was that the deployed model should not run continuously, but only when a tomato is actually present to be classified, with a clear separation in time between successive classification events. Two complementary implementations were built to satisfy this.

Manual-trigger mode performs no camera polling or inference at all while idle; a classification sequence (six views, with a one-second interval between them to allow the tomato to be rotated) is triggered only on an explicit input, and the resulting per-class confidence scores are summed across all six views to produce a single verdict, following the same multi-view fusion principle described in Chapter 7.

Automatic-trigger mode uses a cheap presence-detection check – comparing the saturation and frame-to-frame motion of a downsampled camera frame against a calibrated empty-chamber baseline – running several times per second with no model inference involved. The detection model is invoked only once a tomato has been detected as present *and* stationary for a sustained period (filtering out hand motion during placement), and the system will not re-trigger on the same tomato until the chamber has registered as empty again. In testing, this correctly fired a single classification event per tomato placed, correctly ignored the motion of a hand placing or removing a tomato, and reported that the detection model was active for only a small fraction of total runtime, consistent with the design requirement.

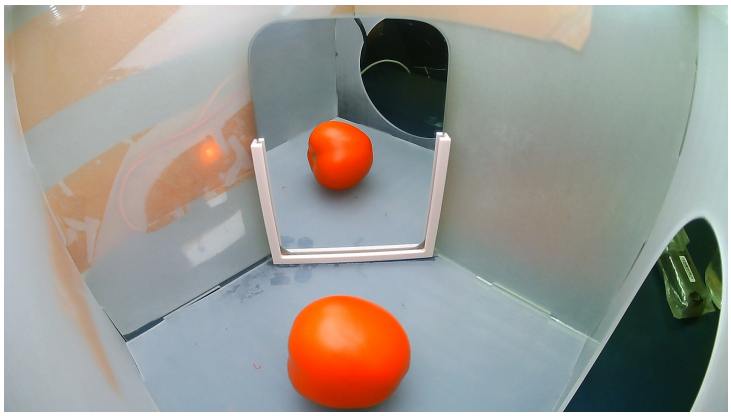


Figure 8.1: A single captured view from a triggered classification event on the Raspberry Pi, correctly localizing both the direct and mirror-view tomato instances.

9. Results

This chapter consolidates the key quantitative results from the preceding chapters.

9.1 Diagnosis of the Stage-1 Model

Table 9.1: Summary of diagnostic findings, Chapter 4

Test	**Finding**
Live empty-chamber test	Confident false-positive detection ("unripe", 0.97 confidence) with no tomato present
Model-size comparison (YOLO11n vs. YOLO11s)	Both failed similarly on ripe-class chamber images (20.8% vs. 18.8% accuracy); rules out model capacity as the cause
Black-background re-test	Ripe accuracy dropped to 0% (from 20.8%) when background was artificially matched to training; rules out background mismatch as the sole cause
Visual colour comparison	Training "ripe" images are dull orange/amber; chamber-captured ripe tomatoes are vivid saturated red – a colour only ever labelled "overripe" in training

9.2 Final Model Performance

Table 9.2: Final three-class model – summary of all reported evaluations

Evaluation	**Scope**	**Result**
Same-session validation (by-tomato split)	mAP50-95	0.944
Held-out, cross-session – ripe	3 tomatoes	3/3 correct (18/18 frames)
Held-out, cross-session – unripe	6 tomatoes	6/6 correct (34/36 frames)
Held-out, cross-session – overripe	*[pending]*	*[to be inserted]*

9.3 Edge Deployment Performance

Table 9.3: Raspberry Pi 4 deployment summary

Metric	**Value**
Exported model format	NCNN, FP32
Model size	5.2 MB (PyTorch) / ≈10 MB (NCNN)
PC vs. Pi prediction parity (7 held-out frames)	7/7 identical classifications
Mean inference latency (4 threads)	362 ms
INT8 alternative	rejected (slower and less accurate)

10. Conclusion and Future Work

Stage-2 began by establishing that the Stage-1 YOLO model's near-perfect validation metric did not reflect real deployment performance, and traced this to a specific, demonstrable cause: a colour-and-lighting mismatch between black-background training data and real chamber footage, compounded by a localization mismatch in object scale and framing. This diagnosis directly shaped the rest of the stage's work – an automated annotation pipeline capable of labelling full, real-background chamber photographs was built using SAM2, a three-class chamber-captured dataset (including the logistically difficult overripe class) was collected and correctly split by tomato to avoid validation leakage, and the resulting fine-tuned model was evaluated not only on a same-session split but on a genuinely independent, cross-session test set – which itself exposed and led to the correction of a further generalization gap on the ripe class.

The final model was deployed on a Raspberry Pi 4, with its predictions verified to match the original PyTorch model exactly, its inference speed benchmarked and tuned, an INT8 quantization alternative evaluated and rejected on both speed and accuracy grounds, and an event-triggered inference design built and validated so that the model performs classification only when a tomato is actually present.

Several items remain open for continued work.

- **Overripe generalization testing**, following the same cross-session protocol used for ripe and unripe, is in progress; results will be incorporated into Chapter 7 once complete.
- **Sustained-load and thermal testing** on the Raspberry Pi, to confirm inference latency remains stable over extended continuous operation rather than only short benchmark runs, has not yet been carried out.
- **A dedicated AI-accelerator camera module (reCamera)** was investigated as a higher-performance alternative to the Raspberry Pi, but a decision on whether to pursue this is pending a defined throughput requirement (tomatoes processed per minute) from the project's practical deployment goals.
- **Testing with the chamber's dedicated camera module**, rather than a general-purpose USB webcam, to assess whether a different camera's colour and lens char-

acteristics reintroduce a version of the domain-gap problem diagnosed in Chapter 4, specific to that hardware.

- **Integration with the physical sorting mechanism** – connecting the triggered-inference classification output to the Stage-1 prototype's feeder and sorting actuators – remains to be carried out for a fully end-to-end automated system.